Detecting Common Road Users in Philippine Traffic: A Comparative YOLOv26 Study

Cloie Sofia R. Dela Cruz[1,+], Ralf Cedric T. Zabala[1,+], John Paul Q. Tomas[1,*], Bonifacio T. Doma[2,3,4,+]

[1]School of Information Technology, Mapúa University, Manila, Philippines. [2]School of Chemical, Biological, and Materials Engineering and Sciences, Mapúa University, Manila, Philippines. [3]School of Graduate Studies, Mapúa University, Manila, Philippines. [4]School of Health Sciences, Mapúa University, Manila, Philippines. *jpqtomas@mapua.edu.ph; btdoma@mapua.edu.ph

Abstract

Reliable detection of road users in Philippine traffic is important for traffic monitoring and other safety-oriented computer vision applications. This study compared three YOLOv26 training configurations for detecting seven common road-user classes using a balanced dataset of 1,310 traffic-scene images. The models were trained and evaluated in Google Colab while varying optimizer, batch behavior, learning rate, and epoch count. Performance was assessed on the validation split using mAP50, precision, recall, and F1 score. Among the tested configurations, Model_C produced the strongest aggregate validation performance, achieving an mAP50 of 0.6178, precision of 0.6500, recall of 0.5911, and F1 score of 0.6192. Class-wise results remained uneven: Jeepney, SUV, and Truck were comparatively strong, whereas Tricycle was persistently difficult across all runs. The findings indicate that training configuration materially affects detection quality on this dataset, but the current comparison should be interpreted as a comparative validation study rather than a final deployment benchmark because multiple hyperparameters changed simultaneously and the held-out test split was not yet used for the final model selection.

Introduction

Road-user detection is a foundational computer-vision task because it supports scene understanding, surveillance, tracking, and intelligent transportation analysis [1,2]. In traffic environments, this task is especially challenging because road scenes are crowded, dynamic, and affected by occlusion, scale variation, illumination changes, and camera-related distortion [3–5]. The problem becomes more difficult when several road-user categories appear in the same frame and differ substantially in size, appearance, and frequency.

The Philippine traffic context provides a meaningful setting for this problem. Urban scenes may contain bicycles, cars, jeepneys, pedestrians, sport utility vehicles, tricycles, and trucks within the same view. This heterogeneity increases detection difficulty and creates a useful benchmark for assessing how a detector performs across classes with different visual signatures and class frequencies. In practical terms, stronger detection of these road users may support later work in traffic monitoring, counting, surveillance, and safety-oriented analytics.

Real-time object detectors are well suited to this domain because they perform localization and class prediction in a single pipeline [1]. The YOLO family remains one of the most widely used examples of this design approach, and prior studies show that YOLO-based methods remain relevant to traffic-oriented perception because of their balance between speed and accuracy [1,3]. More recent traffic-focused studies likewise show that urban and fish-eye traffic scenes remain difficult because of small objects, clutter, distortion, and class confusion [4,5]. In this study, YOLOv26 was selected as the experimental detector and trained under three configurations to examine how optimizer choice, batch behavior, learning rate, and epoch count influence detection performance on a Philippine traffic dataset.

This paper does not propose a new detector architecture. Instead, it presents a comparative evaluation of three YOLOv26 training configurations using a balanced dataset of common road users in Philippine traffic scenes. The study aims to identify which tested configuration provides the strongest validation performance and to interpret the observed differences in aggregate and class-specific metrics. In this way, the paper provides an initial benchmark that can support later validation, refinement, and more deployment-oriented evaluation.

Methods

Experimental Environment

All training and evaluation procedures were executed in Google Colab using a Python 3 environment with the Ultralytics package (version 8.4.18) and an NVIDIA Tesla T4 GPU. Google Drive was mounted inside the Colab runtime and used to access the dataset directory and store training outputs. This setup allowed the experiment to use cloud GPU acceleration while preserving the dataset and run artifacts in a shared project workspace.

Conceptual Framework

The conceptual framework of the study is shown in Figure 1. Traffic-scene images from the balanced dataset served as the study input. These images were processed through three YOLOv26 training configurations, each of which produced validation

metrics, class-wise summaries, and diagnostic figures. The comparative interpretation stage used these outputs to determine which tested configuration provided the strongest overall performance and where class-specific weaknesses remained.

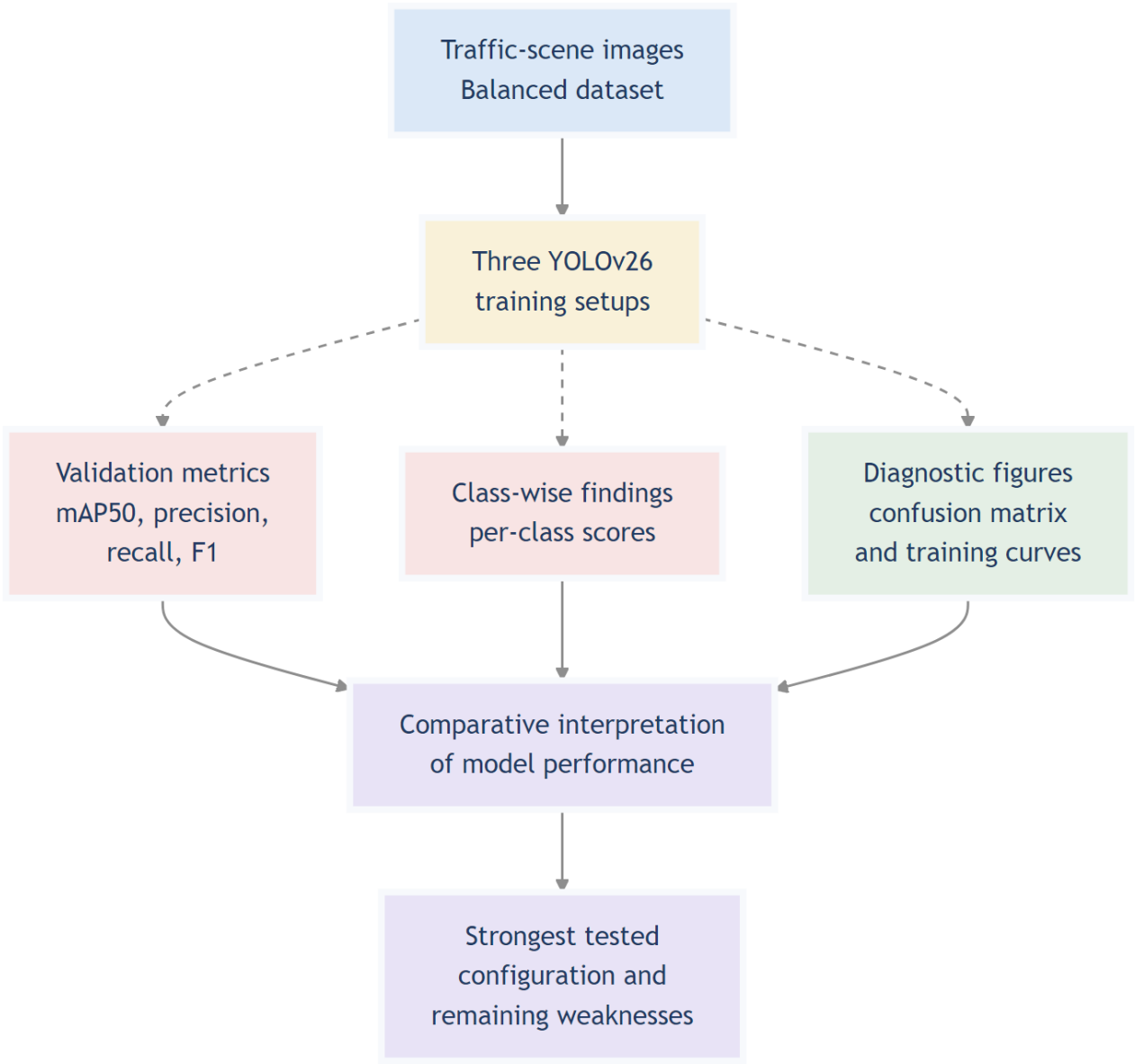


Figure 1. Conceptual framework for comparative YOLOv26 evaluation

Dataset

The dataset originated from video footage of Philippine traffic scenes. Frames were extracted and annotated through a Roboflow-based workflow, then exported in YOLO26 format for model training and evaluation. According to the Roboflow export record, the dataset was exported on February 28, 2026, with auto-orientation and resizing to 640 x 640 applied during preprocessing. For the main experiment, we used the balanced version of this dataset stored in the Balanced directory of the project workspace. The dataset contained seven object classes: Bicycle, Car, Jeepney, Pedestrian, SUV, Tricycle, and Truck. Based on the notebook outputs, the dataset comprised 1,310 images in total, partitioned into 1,010 training images, 154 validation images, and 146 test images. The training set included both original and augmented images. Specifically, 705 training images were original samples and 305 were augmentation-derived samples. The notebook also reported residual class

imbalance in the training annotations, with class counts ranging from 539 to 1,581 and an imbalance ratio of 2.93, indicating that the balanced dataset improved but did not eliminate class frequency differences.



Figure 2. Sample annotated images from the balanced traffic dataset

Data Configuration

The dataset used a standard YOLO directory structure with train/images, valid/images, and test/images paired with corresponding label folders. During the experiment, the dataset YAML file was configured to point to the balanced dataset root and to use relative paths for the train, validation, and test splits. This ensured compatibility with the Ultralytics training and validation workflow inside the Colab environment.

Model And Training Configurations

All three runs used the yolo26n.pt weights as the starting model. The input image size was fixed at 640 for all runs, but the optimizer, number of epochs, batch behavior, and initial learning rate varied across configurations.

- Model_A: 25 epochs, AdamW optimizer, batch size 4, learning rate 0.01

- Model_B: 30 epochs, SGD optimizer, batch size 20, learning rate 0.001
- Model_C: 40 epochs, automatic optimizer selection, automatic batch sizing, learning rate 0.0001

Each model was trained through the Ultralytics train procedure with outputs stored under the shared detect project directory. The notebook retained the default training plots and confusion matrices produced by the framework for later interpretation.

Study Workflow

Figure 3 summarizes the operational workflow followed in the experiment. The process began with dataset preparation and configuration in Google Drive, followed by Colab-based environment setup and Ultralytics training. The three model runs were then validated on the validation split, after which aggregate metrics, class-wise summaries, confusion matrices, and training-dynamics plots were prepared for comparative reporting.

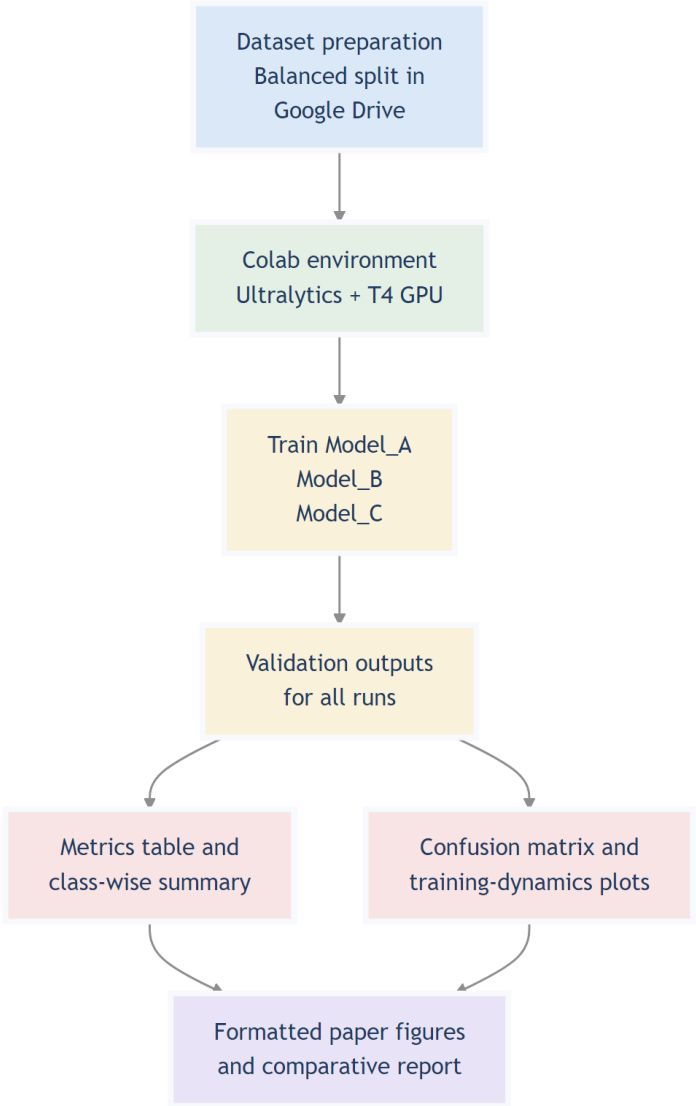


Figure 3. Experimental workflow for YOLOv26 model comparison

Evaluation Procedure

After training, each model was evaluated using the Ultralytics validation routine on the validation split. Aggregate metrics recorded in the notebook were mAP50, precision, recall, and F1 score. The F1 score was computed from precision and recall. In addition to aggregate metrics, the workflow also extracted per-class precision, recall, AP50, and F1 score values. These outputs were then assembled into comparison tables and visualization-ready summaries for manuscript reporting.

Although the dataset contained a separate test split, the current reported results are based on the validation outputs documented in the notebook. Accordingly, the paper treats the reported values as comparative validation results rather than as a final deployment-grade benchmark.

Results

Three YOLOv26 configurations were evaluated on the same balanced dataset and compared using aggregate validation metrics. Across the tested runs, Model_C produced the strongest overall result, followed by Model_A, while Model_B performed worst on all reported aggregate measures. Table 1 summarizes the comparative performance of the three models.

Table 1. Comparative Performance of YOLOv26 Model Configurations

Model	Epochs	Optimizer	Batch Size	Learning Rate	mAP50	Precision	Recall	F1 Score
Model_A	25	AdamW	4	0.01	0.6034	0.6682	0.5274	0.5895
Model_B	30	SGD	20	0.001	0.4858	0.5007	0.5029	0.5018
Model_C	40	auto	auto-batch	0.0001	0.6178	0.6500	0.5911	0.6192

Table 1 reports mAP50, precision, recall, and F1 score for the three tested YOLOv26 configurations. Model_C achieved the highest mAP50 (0.6178), recall (0.5911), and F1 score (0.6192), while Model_A achieved the highest precision (0.6682). Model_B was lowest on mAP50, precision, and F1 score. These aggregate results show that the three tested configurations produced clearly different validation outcomes on the same dataset.

Class-wise performance remained uneven across the seven categories. Jeepney was one of the strongest classes across all three models, with F1 scores of 0.7456 for Model_A, 0.6455 for Model_B, and 0.7299 for Model_C. SUV and Truck were also comparatively stable for Model_C, reaching F1 scores of 0.6607 and 0.6246, respectively. By contrast, Tricycle was the weakest class in all three runs, with F1 scores of 0.3659, 0.2027, and 0.4168 for Models A, B, and C. Pedestrian and Bicycle showed intermediate performance levels.

Confusion Matrix

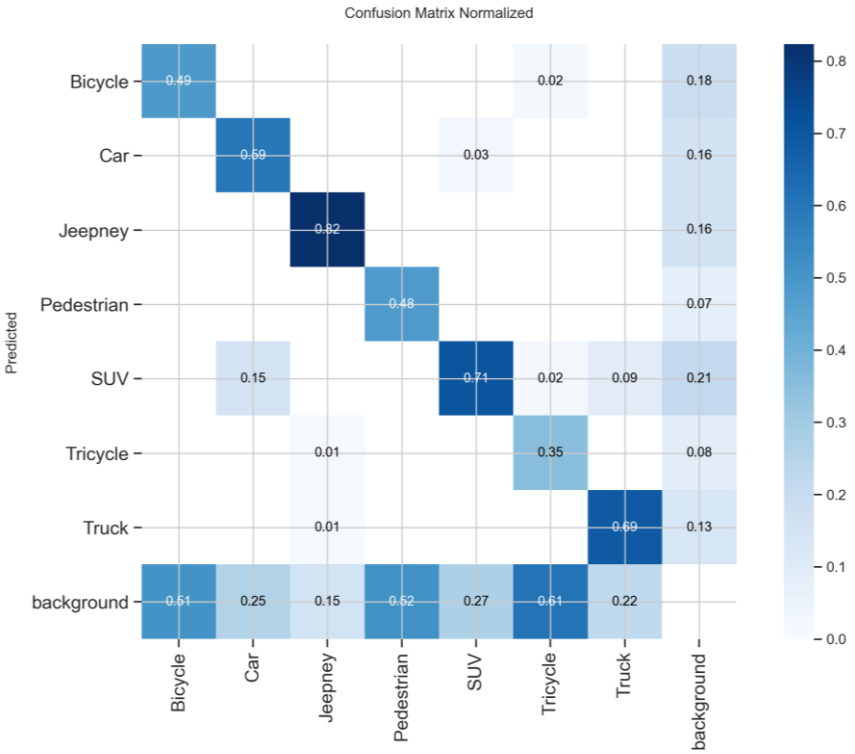


Figure 4. Normalized confusion matrix for Model_C

The confusion matrix presents the normalized confusion matrix for Model_C, the best-performing tested configuration. The matrix shows stronger concentration along the diagonal for classes with better class-wise scores and visibly broader error regions for weaker categories. The weaker class-wise F1 values reported for Tricycle, Bicycle, and Pedestrian are consistent with the more diffuse error pattern outside the diagonal.

Training Dynamics

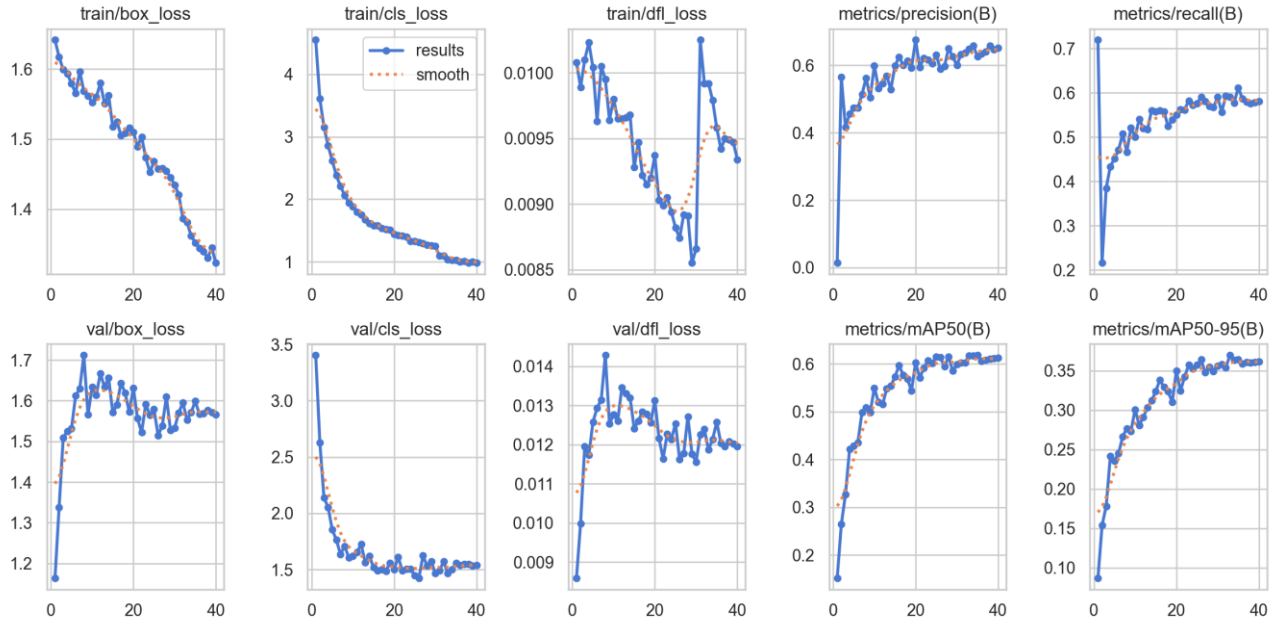


Figure 5. Training and validation metrics across epochs for Model_C

The metric shows the training-dynamics curves produced by the Ultralytics run artifacts for Model_C. The figure summarizes the evolution of loss and metric trends across epochs for the selected configuration and provides a visual record of the run used as the strongest comparative result in this study.

Results-Synthesis Diagram

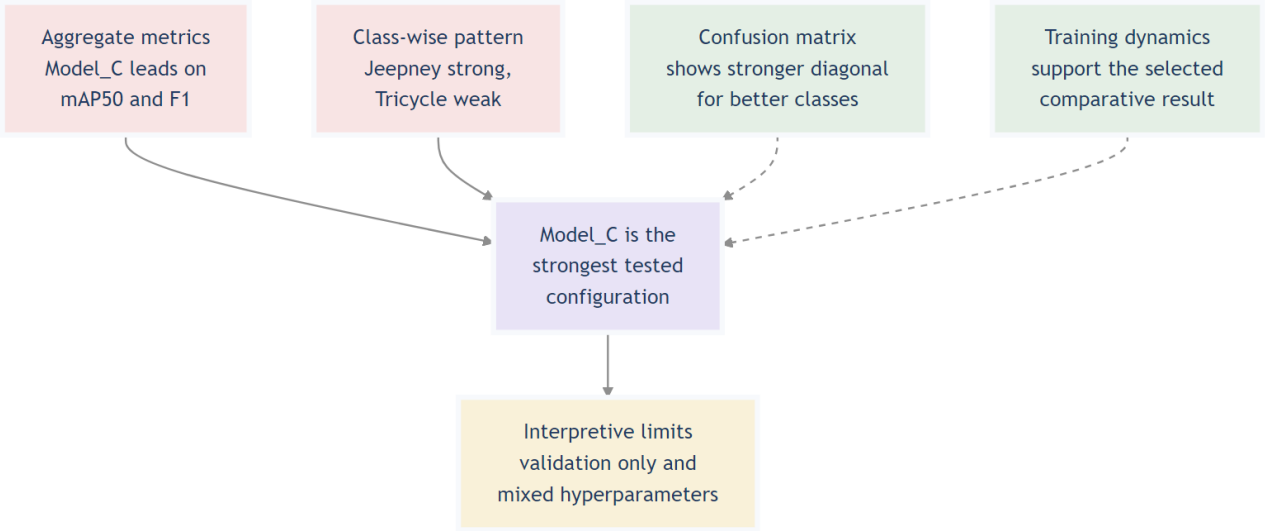


Figure 6. Synthesis of comparative results across evaluation metrics

The results synthesizes the main strands of evidence used in the comparison. Aggregate metrics, class-wise findings, the confusion matrix, and the training-dynamics curves all point to Model_C as the strongest tested configuration in the present validation-based experiment.

Discussion

The results show that training configuration choice had a meaningful effect on detection quality, but the present comparison should be interpreted carefully because several hyperparameters were changed at the same time. Model_C differed from the other two runs in optimizer selection, epoch count, learning rate, and batch behavior. As a result, its stronger aggregate performance cannot be attributed to a single factor with certainty. Nevertheless, the outcome still suggests that the Model_C configuration produced a more favorable precision-recall balance on this dataset than the settings used in Models A and B.

One useful contrast is the relationship between Model_A and Model_C. Model_A achieved the highest precision, indicating more conservative detection behavior and fewer false positives overall. However, its lower recall reduced its final F1 score. Model_C maintained competitive precision while improving recall, which produced the highest F1 score among the three runs. A cautious interpretation is that the smaller learning rate, longer training schedule, and automatic batch handling in

Model_C may have supported a more favorable trade-off between missed detections and false positives on this dataset. Because these factors changed together, the paper should treat this as a configuration-level interpretation rather than a single-hyperparameter explanation.

Model_B provides the clearest negative contrast. Its lower performance across all aggregate metrics suggests that the tested combination of SGD, batch size 20, 30 epochs, and learning rate 0.001 was less compatible with the present data and training setup than the other two configurations. This result does not imply that SGD is generally inferior; rather, it indicates that this specific parameter combination was the weakest among the tested alternatives. The paper should therefore avoid causal claims that isolate optimizer choice alone.

The class-wise results also reveal important dataset-level and task-level challenges. Tricycle was the weakest class across all three models, with Model_C improving the class only to an F1 score of 0.4168. This persistent weakness suggests that Tricycle instances may be harder to detect because of their smaller visual footprint, structural variability, overlap with other vehicles, or lower representation quality in the dataset. Pedestrian and Bicycle also showed lower scores than the strongest vehicle classes, which is consistent with the broader traffic-detection literature where small, partially occluded, or visually inconsistent objects remain difficult to detect reliably[4,5]. By contrast, Jeepney, SUV, and Truck were comparatively easier for the detector, likely because of stronger shape cues and more stable visual identity in the training images.

Another relevant consideration is the composition of the balanced dataset itself. Although the dataset was augmented to improve class coverage, the notebook outputs show that the training split still retained uneven annotation counts across classes, with an imbalance ratio of 2.93 even after balancing. The training set also combined 705 original images with 305 augmented images, which means some of the apparent balancing benefit may come from synthetic variation rather than from large amounts of new real-world data. This does not invalidate the experiment, but it does mean that improved performance should be interpreted as performance on the current balanced dataset formulation rather than as evidence of universal robustness.

The diagnostic figures help reinforce these interpretations. The normalized confusion matrix indicates that stronger classes are associated with cleaner diagonal concentration, whereas weaker classes show broader confusion patterns. The training-dynamics curves indicate that the selected configuration converged to the best reported comparative result within the present validation-based study. Together, these visuals complement the tabulated metrics by showing that the observed performance differences were not limited to a single summary value.

Finally, the study has several practical limitations. The best reported result is based on validation metrics rather than a final held-out test comparison. The three configurations are not one-variable-at-a-time experiments, so fine-grained causal interpretation remains limited. In addition, the paper evaluates image-based detection rather than end-to-end video analytics or deployment behavior. For these reasons, Model_C should be treated as the strongest tested configuration in this experiment, but not yet as a deployment-ready solution. Future work should separate hyperparameter effects more cleanly, strengthen label-level and class-specific error analysis, and validate the selected configuration on a stricter final test protocol.

References

1. Redmon J, Divvala S, Girshick R, Farhadi A. You only look once: Unified, real-time object detection. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition; 2016 Jun; Las Vegas, NV, USA. p. 779-788. Available from: https://openaccess.thecvf.com/content_cvpr_2016/html/Redmon_You_Only_Look_CVPR_2016_paper.html
2. Liu L, Ouyang W, Wang X, Fieguth P, Chen J, Liu X, Pietikainen M. Deep learning for generic object detection: A survey. Int J Comput Vis. 2020;128:261-318. Available from: https://doi.org/10.1007/s11263-019-01247-4
3. Wu D, Liao MW, Zhang WT, Wang XG, Bai X, Cheng WQ, Liu WY. YOLOP: You only look once for panoptic driving perception. Mach Intell Res. 2022;19:550-562. Available from: https://doi.org/10.1007/s11633-022-1339-y
4. Luo X, Cui Z, Su F. FE-Det: An effective traffic object detection framework for fish-eye cameras. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops; 2024 Jun; Seattle, WA, USA. p. 7091-7099. Available from: https://openaccess.thecvf.com/content/CVPR2024W/AICity/html/Luo_FE-Det_An_Effective_Traffic_Object_Detection_Framework_for_Fish-Eye_Cameras_CVPRW_2024_paper.html
5. Soudeep S, Jahin MA, Mridha MF. Interpretable dynamic graph neural networks for small occluded object detection and tracking [Internet]. arXiv [cs.CV]. 2024 [cited 2026 Mar 17]. Available from: https://arxiv.org/abs/2411.17251